Yes. I would rebuild the preparation bank around **your actual resume + the exact email you already sent Mac**, not around your friend's Ticketmaster project.

There are several things in the sample bank you **must not use** because they are not yours: the Ticketmaster experiment, Ehban conversations, Norman Lear Center examples, Kubernetes contribution, “four other candidates,” feed-health dashboard, SQLite prototype, or anything else your friend built. Tomorrow, Mac should hear one completely consistent Sahaj story.

Your strongest real evidence is substantial: USC Dornsife Azure SQL/FastAPI work, 1.2s→350ms latency reduction, fault-tolerant async processing, Grad8's 10K+ requests/day production system, Aasma's API/ETL work and 2M+ record synchronization, plus AWS/Azure/database experience.

1. First: fix the story in your head

Your positioning tomorrow should be:

> **“I'm a backend/data engineer who likes turning messy, ambiguous workflows into reliable systems. My strongest experience is APIs, SQL/data systems, pipelines, reliability and performance. What attracts me to Sky Strike is that those systems directly affect business decisions.”**

Do **not** position yourself primarily as:

> AI engineer
> ML researcher
> LLM engineer

Those are useful secondary capabilities. Your resume does have AI/ML experience, but Sky Strike's immediate need is much closer to your backend/data experience. Your listed technical base includes Python, SQL, PostgreSQL, AWS, Azure, distributed systems, ETL and system design.

---

2. Important email/resume consistency check

This is the most important preparation before anything else.

What you wrote Mac

You said:

> “Python/FastAPI + SQL production systems processing 10K+ records”

Your resume supports **Python/FastAPI + Azure SQL** at USC Dornsife, but the Dornsife metrics shown are **500+ users and 1K+ AI jobs**, not 10K records.

The **10K+ metric** on your resume belongs to Grad8:

> “handling 10K+ grading requests/day.”

So if Mac asks about it, do **not** double down on “10K records at Dornsife.”

Say:

> “Just to clarify the metrics, at Dornsife the platform supported 500+ users and the async orchestration processed 1K+ AI jobs. The 10K+ production-volume metric is from Grad8, where the backend handled 10K+ grading requests per day.”

Perfectly fine.

You also wrote “reconciliation pipelines”

That exact phrase isn't on this resume.

If you genuinely built reconciliation/validation logic, be ready to describe **exactly what was compared, how inconsistencies were detected, and what happened when they disagreed**.

If you cannot explain a specific implementation, don't keep repeating “reconciliation pipelines.” Instead say:

> “data validation and consistency checks”

Your 1.2s → 350ms claim is fully supported

That is one of your best stories. Your resume says you reduced dashboard latency from 1.2 seconds to 350ms through query and indexing optimization.

One resume issue Mac could notice

Your summary says you're **“pursuing an MSCS”**, while the education section says August 2024–May 2026.

Since it's September 2026, if asked:

> “I completed the degree in May. The word 'pursuing' in the summary is outdated wording on that resume version.”

Don't overexplain.

---

3. The framework to use for almost every Mac question

Memorize this:

> **Goal → Current process → Constraints → Evidence → Options → Tradeoffs → Small test → Measurement → Recommendation**

But don't recite it.

Suppose Mac says:

> “We're paying for two data providers. Which one should we keep?”

Do **not** immediately answer.

Start:

> “Before choosing one, I'd want to understand what decision we're using that data to make and what a bad or missing value costs us.”

Then work through the problem.

The two sentences I want you using repeatedly tomorrow are:

> **“Before deciding, I'd first want to understand what outcome we're optimizing for.”**

and:

> **“My first hypothesis would be X, but I wouldn't want to assume that. I'd validate it by checking Y.”**

---

4. Your likely interview flow tomorrow

I would expect roughly:

First 3–5 minutes: greeting, background, why you contacted Sky Strike.

Next 20–30 minutes: Mac gives you ambiguous business scenarios and adds information as you reason.

Next 5–10 minutes: discussion about how your prior experience maps to their problems.

Last few minutes: your questions and next steps.

The key is that Mac explicitly said you will **work through problems together**.

Don't deliver three-minute monologues.

A good rhythm is:

> Mac gives problem → you clarify → Mac gives context → you reason → Mac challenges assumption → you adjust → you recommend.

---

5. Opening questions

1. Tell me about yourself.

> “I recently completed my M.S. in Computer Science at USC. Most of my recent work has been backend and data-oriented engineering—APIs, SQL systems, pipelines, reliability and performance.
>
> At USC Dornsife I worked on an Azure SQL/FastAPI platform where I reduced dashboard latency from 1.2 seconds to 350 milliseconds and worked on fault-tolerant asynchronous processing. Before that, at Grad8 I worked on an AWS backend handling 10K+ grading requests per day.
>
> What attracted me to Sky Strike is that the engineering is very close to the actual business decision. You're not just building software—you can see whether better data or automation actually helps the desk.”

Every factual component is on your resume.

---

2. Why Sky Strike?

> “The biggest reason is how close engineering is to the business. If I improve data freshness, reliability or an internal workflow here, I can understand directly whether it helped someone make a better or faster decision.
>
> The other part is the technical fit. External APIs, data pipelines, SQL, backend services and automation are very close to work I've already done.”

---

3. What do you understand about our business?

Keep it basic.

> “My understanding is that Sky Strike buys ticket inventory and manages/resells that inventory in the secondary market. Unlike many assets, a ticket has a hard expiration date, so pricing, timing, inventory and demand all matter.
>
> From an engineering perspective, that means the quality and freshness of market information can directly affect business decisions.”

Don't pretend you understand their proprietary strategy.

---

4. What do you think this engineering role is supposed to accomplish?

> “I see the immediate goal as making the information the desk depends on more reliable and easier to act on—external data coming in correctly, clean storage, useful internal services and visibility when something breaks.
>
> Once that foundation is trustworthy, more sophisticated automation and predictive work becomes much safer.”

This matches the email you already sent him.

---

5. Why a small company?

> “I like short feedback loops. At a small company I can talk directly with the person using something, understand whether it actually solved the problem, and adjust quickly.
>
> I also like owning something end to end instead of only seeing one small component.”

---

6. You haven't worked in ticket trading. Why should that not concern me?

> “I definitely don't want to pretend I know the ticket market better than your traders.
>
> What I can bring immediately is experience with APIs, external data, SQL, backend systems, reliability and performance. I'd learn the business from the people making the decisions and apply the engineering patterns I already know.”

Strong answer.

---

6. Questions Mac can ask specifically because of your email

7. You said you could make our data ingestion more reliable. How?

Do not start technical.

> “I'd first understand what failures you're actually seeing today. Missing pulls? Incorrect values? Slow updates? Duplicate data?
>
> Once I know the failure modes, I'd measure them. Then I'd improve the highest-impact ones and make failures visible so stale or incorrect data isn't silently treated as current.”

If he asks technical follow-up:

> “That could mean validation, retries, logging, timestamps, freshness checks and controlled fallbacks depending on the failure.”

---

8. You said you'd build faster decision tools. What does that mean?

> “That was intentionally a hypothesis based on what I understood from the role.
>
> Before building a dashboard, I'd sit with the trader and understand what information they currently gather manually before making a decision. Then I'd identify which pieces are slow, repetitive or hard to trust.
>
> The interface should come from that workflow rather than me deciding upfront what a trader needs.”

Excellent correction if the email sounded presumptive.

---

9. You said you'd automate purchasing. What exactly would you automate?

This is a likely challenge.

> “I actually wouldn't start by automating purchasing itself.
>
> I'd first observe the purchasing workflow and separate deterministic repetitive steps from judgment calls. I'd probably begin with information gathering or decision support.
>
> Anything involving actual money should initially have clear limits and human approval. Automation authority can expand only after the underlying data and workflow are trustworthy.”

Very strong.

---

10. Why did your subject say “make more trades with less manual work”?

> “I was trying to frame engineering in terms of business outcomes rather than technologies.

> I don't actually know yet whether trade volume or manual work is your biggest bottleneck. Those were hypotheses from the role description. That's one of the things I'd want to learn from you."

Perfect.

---

7. Core business scenarios — these are the most important

11. Two providers give us different values. Which one do you trust?

> "First I'd verify that we're comparing the same thing—same event, timestamp, inventory definition, fees and other relevant conditions.
>
> If it's a genuine disagreement, I'd compare each provider historically against outcomes or another trusted reference.
>
> I wouldn't automatically decide one source is always correct. They may have different strengths depending on the data."

---

12. Provider A is cheap but unreliable. Provider B costs much more.

> "I'd quantify the cost of unreliability before comparing subscription prices.
>
> If missing or wrong data can lead to a bad purchasing decision, a more expensive provider may easily pay for itself.
>
> I'd run both against the same sample, measure coverage, accuracy, freshness and failure rate, then estimate what those differences mean financially."

---

13. How would you compare two vendors?

> “I'd define ‘better’ first.
>
> Probably some combination of coverage, accuracy, freshness, reliability, latency, cost and integration effort.
>
> Then I'd run both on the same events and time windows, collect enough observations, and compare them objectively before making a recommendation.”

---

14. An API sometimes gives us zero. What do you do?

> “First I'd establish whether zero is a legitimate business value or represents missing/broken data.
>
> I'd inspect surrounding historical values, response status, timestamp and potentially another source.
>
> Once we understand what a legitimate zero looks like, we can measure the actual failure rate rather than guessing.”

This is exactly the right Mac style.

---

15. What if two marketplaces disagree?

> “I'd first make sure the data is comparable. For example, whether they're showing the same event, section, quantity, fees and timestamp.
>
> If they still disagree, I wouldn't necessarily call one wrong. It could reflect different inventory or liquidity.
>
> I'd look at historical reliability and potentially retain both values with source attribution.”

---

16. We have tons of data. How do we figure out what matters?

> “I'd start with the decisions the traders are trying to improve.
>
> For each dataset I'd ask: what decision could this change?

> Then I'd test whether the information adds something we don't already have. Data without a decision attached to it can become expensive storage rather than business value."

Excellent founder answer.

---

17. Someone spends four hours every day on a manual process. What do you do?

> "I'd watch them perform the workflow before automating anything.
>
> I'd break it into repetitive deterministic steps and judgment-heavy steps, estimate the cost of mistakes, and automate the safest high-volume portion first.
>
> I'd measure time saved and error rate before expanding automation."

---

18. How do you decide whether automation is worth building?

> "Frequency × time spent × cost of mistakes × engineering effort.
>
> A task taking five minutes once a month probably isn't worth automating. A task taking hours every day and following predictable rules probably is."

---

19. We have ten projects. Which one comes first?

> "I'd look at expected business impact, urgency, engineering effort and uncertainty.
>
> Something affecting revenue, preventing expensive errors or removing hours of recurring work would normally outrank a nice-to-have feature.
>
> For highly uncertain ideas, I'd first run the smallest experiment that can tell us whether they're worth building."

---

20. Historical data or real-time data—which is more important?

> “It depends on the decision.
>
> If someone is deciding whether to purchase something right now, freshness may dominate.
>
> If we're evaluating strategy, seasonality or building forecasting models, historical depth becomes more important.
>
> I'd let the business decision define the latency requirement.”

---

21. What if real-time data is very expensive?

> “I'd question whether every data point actually needs real-time updates.
>
> We might have high-priority events requiring frequent polling and lower-priority events that can update less often.
>
> I'd match refresh frequency to the economic value of freshness rather than applying one rate to everything.”

Very good.

---

22. How do you evaluate whether a new data source is worth buying?

> “I'd start with what new decision it enables or improves.
>
> Then I'd try to get a trial/sample, compare coverage and quality against what we already have, and run it through a few situations we understand.
>
> If it doesn't change a useful decision, more data by itself probably isn't worth paying for.”

---

23. What if the new provider has better data but a difficult integration?

> “I'd compare the ongoing value against both implementation cost and maintenance cost.
>
> Integration complexity is a real cost, but I wouldn't reject the provider only because it's technically inconvenient if the data creates meaningful business value.”

---

24. We want to know whether something we built is helping. How?

> “Define the success metric before building it.
>
> Depending on the tool, that could be time saved, fewer failures, lower data latency, better coverage, fewer bad decisions or some revenue-related measure.
>
> Without a baseline, it's difficult to know whether the software improved anything.”

---

25. Something seems like an outlier. Delete it?

> “Not immediately.
>
> I'd first determine whether it's a data error or a real market event.
>
> In a trading-related business, unusual values can sometimes contain more information than normal ones. I'd preserve the raw value while investigating it rather than silently cleaning it away.”

---

26. Data is missing. Would you fill it in?

> “Only if we understand why it's missing and the downstream use.
>
> I'd rather explicitly represent missing data than invent a value that looks legitimate.
>
> If we do impute something for analysis, I'd keep track of which values were estimated versus observed.”

---

27. What if an external service goes down during an important period?

> “First, don't hide the failure.
>
> I'd keep the last known good data but clearly mark it stale. Depending on the business importance, I'd retry, alert someone and possibly use a fallback source.
>
> The trader should know the confidence and freshness of what they're looking at.”

---

28. Should we keep two providers for redundancy?

> “Potentially, but I'd quantify whether the redundancy is worth its cost.
>
> If losing one provider during important periods creates significant financial risk, redundancy could be justified.
>
> If both sources usually fail or succeed together, we're paying twice without getting much resilience.”

---

29. Build something internally or buy a tool?

> “I'd compare strategic value, customization needs, cost, time-to-value and ongoing maintenance.
>
> If it's generic infrastructure, buying may be faster. If the workflow itself is part of Sky Strike's competitive advantage, owning it can make much more sense.”

Excellent small-company thinking.

---

30. What if Mac gives you a constraint that destroys your idea?

Say:

> “That changes my recommendation.”

Then adapt.

Don't defend your original solution because you already said it.

Mac is probably looking for exactly this.

---

8. Resume questions you should absolutely expect

Your resume is dense. He may choose something and ask you to explain it without technical buzzwords.

31. Tell me about the 1.2s → 350ms improvement.

> “The application was querying a multi-tenant Azure SQL database and some dashboard requests were slower than they needed to be.
>
> I looked at the query patterns rather than immediately changing application code, identified inefficient access patterns, and improved the queries/indexing around the most common requests.
>
> That reduced dashboard latency from roughly 1.2 seconds to 350 milliseconds.”

Supported directly by your resume.

If he asks **business impact**:

> “The main impact was reducing the waiting time for users repeatedly interacting with the dashboard. It also taught me to measure the actual bottleneck before optimizing.”

---

32. How does that relate to us?

> “The same principle applies here: before deciding a system needs more infrastructure, I'd identify where the latency actually comes from—provider, network, query, transformation or UI—and fix the part that matters.”

---

33. Explain the fault-tolerant LLM orchestration system without buzzwords.

> “We had multiple AI jobs running concurrently, and one failing job shouldn't block unrelated jobs.
>
> I restructured the workflow around explicit job states and isolated failures so each task could succeed or fail independently.
>
> That improved throughput under concurrent load.”

Your resume reports 1K+ jobs and 3× throughput.

---

34. What did “failure isolation” mean?

> “A failure in one task was handled as that task's failure rather than crashing or corrupting the larger workflow.
>
> The important business principle is the same as with external market feeds: partial failure shouldn't make the entire system unusable.”

---

35. Tell me about the Grad8 system.

> “Grad8 was an AI-assisted grading platform handling text and handwritten submissions.
>
> I worked on the backend and batch-processing pipeline, including secure file ingestion and AWS-backed services. The production backend served 150+ instructors and handled 10K+ grading requests per day.”

Directly supported.

---

36. What did 10K+ mean?

Answer precisely:

> “10K+ grading requests per day.”

Not:

> “10K database records.”

This keeps the email/resume story clean.

---

37. Tell me about your Aasma API experience.

> “At Aasma I worked on large API integrations and ETL workflows using AWS Glue and S3. One focus was identifying inefficiencies in the integration path and improving response times by about 30%.
>
> I also worked on ETL improvements that increased pipeline efficiency.”

The resume gives +68% pipeline efficiency and -20% technical debt.

---

38. Tell me about the 2M+ record system.

> “I worked on a MERN-based resource-management system where data had to stay synchronized across locations. The system dealt with more than 2 million records.
>
> That gave me experience thinking about consistency and synchronization at a scale where incorrect assumptions can create a lot of downstream issues.”

Supported.

---

39. Tell me about working with external APIs.

Pick **Aasma**, not an invented Ticketmaster project.

> “Aasma is probably my strongest example. I worked on large-scale API integrations tied into ETL workflows.
>
> The biggest lesson was that an API call succeeding isn't the same as a data pipeline being correct—you have to think about failures, duplicates, response changes, downstream consistency and observability.”

---

40. Tell me about AWS.

> “I've worked with EC2, S3 and RDS, and at Grad8 the production backend used AWS infrastructure and RDS. I've also used AWS Glue/S3 for ETL work at Aasma.”

Your AWS technologies and relevant experience appear throughout the resume.

Don't turn this into a certification-style AWS answer tomorrow.

---

9. Behavioral questions

41. Tell me about a project you're proud of.

Use **Dornsife**.

> “The USC Dornsife platform is probably my best recent example because I wasn't working on just one isolated feature.
>
> I dealt with database design, backend processing, concurrency and performance. The part I'm most proud of is that the improvements were measurable—dashboard latency went from 1.2 seconds to 350 milliseconds and the job system's throughput improved under concurrent load.”

---

42. Tell me about a difficult debugging problem.

Use a real one you can explain deeply.

A safe framework from Dornsife:

> “One issue was understanding why the dashboard was slow. Instead of assuming the frontend or server needed more resources, I traced the request path and measured where time was being spent.
>
> The database access pattern was a major contributor, so I focused on query/index optimization and then measured the latency again.”

Don't invent specific EXPLAIN plans or indexes unless you actually used them.

---

43. Tell me about messy data.

Use Aasma / Grad8.

> “Aasma involved integrating data through different APIs and ETL workflows, so consistency couldn't be assumed.
>
> My general approach was to normalize input into a consistent internal representation, validate important fields and make problems observable rather than letting malformed data silently propagate.”

---

44. Tell me about unclear requirements.

> “I try to convert unclear requirements into a concrete outcome before designing the system.
>
> I'll ask what decision or workflow we're trying to improve, identify the biggest unknown, and build the smallest thing that tests that assumption.
>
> That avoids spending days solving the wrong problem.”

Perfect for Mac.

---

45. Tell me about learning quickly.

> “My approach is usually to start with primary documentation, make the smallest end-to-end path work, inspect real inputs and outputs, and only then expand.
>
> I've had to do that repeatedly across AWS, Azure, new APIs and different AI systems.”

---

46. Tell me about a disagreement.

Your Aasma backbone story can work, but keep it relevant:

> “I've had situations where engineers preferred different implementation approaches.
>
> I try to bring the conversation back to requirements and evidence. If we can create a small test around reliability, performance or complexity, that usually resolves the disagreement better than arguing from preference.”

If Mac asks for a concrete one, use a real specific Aasma disagreement you remember.

---

47. Tell me about a mistake.

Pick something real—not the friend's Ticketmaster identity bug.

A strong general pattern:

> “I've had situations where I initially focused on the implementation before fully validating an assumption about the data or workflow.
>
> What I've changed is that I now try to test the assumption with real inputs much earlier. It usually saves more time than coding a larger solution and discovering the assumption was wrong later.”

If he demands a specific case, use an actual one you personally experienced.

---

48. How do you work independently?

> “For technical unknowns, I'll research, prototype and debug independently.
>
> For business unknowns, I don't think independence means guessing. If I don't know what outcome the trader wants, I'd rather ask a five-minute question than independently build the wrong system for two days.”

This is one of your strongest answers tomorrow.

---

49. How do you handle changing priorities?

> “I ask what changed and why.

> If a data feed affecting active decisions breaks, that would obviously outrank a dashboard enhancement.
>
> Once the new priority is clear, I'll switch quickly, but I want the reason clear so I'm responding to business impact rather than noise."

---

50. What's your weakness?

> "I naturally like understanding technical problems deeply, and sometimes the technically interesting path isn't the highest-value path.
>
> I've gotten better at asking early: what decision does solving this change? That keeps me from over-engineering something that doesn't materially affect the user or business."

Very appropriate for this company.

---

10. Questions about AI

Your resume is AI-heavy enough that Mac may wonder whether you actually want backend work. It includes LLM systems, multimodal grading and several ML projects.

51. You have a lot of AI experience. Why this backend/data role?

> "The AI work actually made me appreciate the data/backend layer more.
>
> A model is only useful when the data feeding it is reliable and the surrounding system works in production.
>
> I enjoy that infrastructure layer, and that's why this role is attractive to me."

---

52. Would you immediately use ML to predict ticket prices?

> “No.
>
> I'd first understand whether we have enough clean historical data, whether the target we're predicting is well defined, and how the prediction would change a decision.
>
> A sophisticated model on unreliable data is worse than a simple rule using trustworthy data.”

Excellent.

---

53. How do you use AI coding tools?

Since you already mentioned them in your email:

> “I use them heavily for accelerating work—exploring unfamiliar APIs, boilerplate, tests, debugging ideas and implementation alternatives.
>
> But I treat the generated output as untrusted until I verify it through documentation, tests and actual inputs and outputs.
>
> I'm still responsible for the code.”

---

54. What if AI generates something that works but you don't understand?

> “I wouldn't put it into an important production path until I understand what it's doing.
>
> If I can't explain the failure modes, I can't responsibly maintain it.”

---

11. Tough business questions

55. What if I disagree with your recommendation?

> “I'd want to understand what information I'm missing.

My recommendation should change if the evidence or business constraint changes. I'm not attached to being right about the first solution."

---

56. What if we need something tomorrow, not in two weeks?

"Then I'd separate what's necessary tomorrow from what belongs in the durable solution.

I'd ship the smallest safe version that addresses the immediate problem, while making the temporary limitations explicit."

---

57. Speed or correctness?

"It depends on the cost of being wrong.

For an internal visualization, an imperfect first iteration may be fine. For something automatically spending money, the correctness threshold needs to be much higher."

Extremely good Sky Strike answer.

---

58. What if a trader wants something you think is technically bad?

"I'd first understand the underlying need.

If the requested implementation creates a genuine reliability or risk problem, I'd explain the tradeoff and propose an alternative that still solves their business problem.

But I wouldn't reject something just because it's not architecturally elegant."

---

59. What makes software valuable to a trader?

> “Trust and usefulness.
>
> If it's fast but sometimes silently wrong, they eventually stop trusting it.
>
> If it's technically perfect but doesn't improve any decision, they won't use it.
>
> I'd optimize for both.”

Use that one.

---

60. What does reliability mean to you?

> “Not simply uptime.
>
> It means knowing whether the data is current, knowing when something failed, being able to recover, and not presenting stale or incorrect information as trustworthy.”

---

12. Employment / fit questions

61. Are you okay with part-time and hybrid?

> “Yes. I understood the structure when I reached out. I'm in Los Angeles and regular time in Santa Monica works for me.
>
> Especially early on, I think being around the traders would actually help because understanding their decision process is important for building the right tools.”

---

62. Are you interviewing elsewhere?

> “Yes, I am speaking with other companies, but I reached out to you directly because this role is unusually aligned with the work I enjoy—backend systems, data pipelines, reliability and close interaction with the business.”

Don't give company names unless he asks.

---

63. Why should I move you forward?

> “I think my experience maps unusually well to the immediate problem: APIs, data systems, SQL, production reliability and performance.
>
> More importantly, I don't need the problem to arrive as a complete technical specification. I enjoy understanding the workflow first and then figuring out what engineering actually creates value.”

---

64. You are still early career. Why trust you with important systems?

> “I wouldn't expect trust just because of my resume.
>
> I'd earn it by making assumptions explicit, testing against real data, making failures visible, shipping incrementally and asking when something involves business context I don't understand.
>
> I've already worked on production systems with real users and meaningful request volumes, but I would still approach financial workflows carefully.”

---

13. Questions YOU should ask Mac

Don't ask all of these. Pick **2–3 based on the conversation**.

Best first:

> **“Of the problems we discussed today, which one creates the most friction for the desk right now?”**

Best second:

> **“If I joined and six months from now you considered hiring me a great decision, what would I have changed for the business?”**

Another strong one:

> **“Where today do you feel the biggest gap is: getting the data, trusting the data, making it easy to act on, or automating the action?”**

Another:

> **“What's something the team currently does manually that you would most like engineering to remove?”**

Another:

> **“When a trader looks at your internal tools, what's the information they absolutely cannot afford to be wrong?”**

That last one is excellent.

---

14. Your real story bank

You only need about **six experiences** ready. Almost every behavioral question can come from these.

Story	Use it for
USC Dornsife: 1.2s → 350ms	performance, debugging, measurement, SQL
Dornsife: async orchestration / 3× throughput	reliability, concurrency, failure handling
Grad8: 10K+ requests/day	scale, production systems, AWS, ownership
Grad8: 60% turnaround reduction	business impact, automation
Aasma: API + ETL + 30% response improvement	external APIs, pipelines, optimization
Aasma: 2M+ record synchronization	large data, consistency, distributed systems

Those claims are all represented on your resume.

You don't need fifteen stories.

You need six stories you can explain deeply.

---

15. Things you absolutely should NOT say tomorrow

Do not say:

> “I built a Ticketmaster dashboard.”

unless **you personally actually built one**.

Do not mention:

> Ehban told me...
> my friend interviewed...
> I know you asked another candidate...
> the Chief Trader said...
> Feed Health...
> Ticketmaster polling every 15 minutes...
> SQLite prototype...
> Kubernetes contribution...

Those belong to the sample preparation, not your resume.

Also don't casually claim:

> “I processed 10K records at Dornsife.”

Your actual resume metrics are different.

And don't pretend to know exactly how Sky Strike trades.

Say:

> **“My understanding is...”**

rather than:

> **“Your system works like...”**

---

16. One special scenario I want you prepared for

Mac may literally pull a real problem from his company and say something like:

> “Here's something we're dealing with right now…”

At that point, forget rehearsed answers.

Take 2–3 seconds.

Then:

> **“Okay. Before I suggest anything, can I ask you two questions about how this is used today?”**

That alone will help you.

Then ask about:

Who uses it?
What decision depends on it?
How often does the problem occur?
What's the cost if it's wrong?
What information do we currently have?

You don't need all five.

Ask the **one or two questions that materially change the answer**.

---

17. Your 30-second closing

At the very end:

> “Thanks, Mac. This is actually the kind of conversation that made me interested in Sky Strike in the first place. I like that the engineering problem starts with understanding the business rather than just getting a specification.
>
> After working through these problems with you, I'm definitely interested in continuing the process.”

Then:

> **“Is there anything from today's discussion or my background that gives you hesitation about moving me to the next round?”**

That is a strong closing because if he has a concern, you get the opportunity to address it **while you're still on the call**.